Problem Summary

Given two binary tree root and SubRoot, determine whether Subroot is a subtree of root.

A subtree means:

→ there is a node in root where the entire tree starting there is exactly the same as SubRoot

Key Idea

At each node in root, ask: "Is the tree starting here exactly the same as SubRoot?"

If Yes → Return True

If No → try again on: → root.left

→ root.right

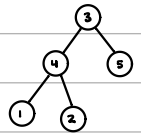
So this problem uses 2 recursive ideas:

1) compare two trees for equality

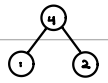
2) Search every node in the main tree

Diagram Examples

root:



SubRoot:



Approach

1) Create helper function SameTree(p,q) to check whether two trees are identical

2) If root is None, return False

3) Check: if SameTree(root, bRoot): return True

4) Otherwise search recursively:

→ isSubtree(root.left, subroot)

→ isSubtree(root.left, subroot)

5) If either side returns True, return True

Time Complexity: $O(n_r * n_{sr})$ → For many nodes in root, we may run SameTree each run can take $O(n)$

Space Complexity: $O(h_r + h_{sr})$ → recursion depth depends on height of root and height of subRoot during SameTree